TrafficLens: YOLO 기반 도로 위 차량·보행자 실시간 인식 시스템

YOLOv8 을 활용한 교통 영상 객체 탐지 및 성능 분석 · 팀 로드비전

팀장 장창근 | **팀원** 반광수 · 지명준 · 장혜원

MBC 아카데미

Oct 12, 2026 · @장창근

1. 서론

본 과제는 TrafficLens 시스템을 통해 YOLOv8 모델로 도로 영상 속 차량과 보행자를 실시간으로 탐지하고, 종류별 개수를 화면에 표시하는 시스템을 구현한다.

1.1 연구 배경

도로 위 차량과 보행자를 사람이 직접 감시하는 방식은 인력과 시간이 많이 들고, 놓치는 상황이 생기기 쉽다. 딥러닝 기반 객체 탐지 기술은 CCTV 나 차량 카메라 영상에서 객체를 자동으로 찾아내 이 문제를 줄일 수 있다. 특히 YOLO 는 속도가 빨라 실시간 처리가 필요한 교통 분야에 적합하다.

1.2 연구 목적

- 사전 학습된 YOLOv8 모델로 차량(승용차, 버스, 트럭, 오토바이)과 보행자를 탐지한다.
- 탐지된 객체를 종류별로 세어 화면에 실시간으로 표시한다.
- 모델 크기별 정확도(mAP)와 처리 속도(FPS)를 비교해 실시간 적용 가능성을 확인한다.

1.3 활용 분야

- 스마트 교차로의 신호 제어와 교통량 조사
- 어린이 보호구역의 보행자 감지 및 경고
- 자율주행 차량의 주변 인식

2. YOLO 개요

YOLO(You Only Look Once)는 이미지를 한 번만 보고 객체의 위치와 종류를 동시에 예측하는 1 단계(one-stage) 객체 탐지 모델이다.

2.1 동작 원리

1. 입력 이미지를 일정한 크기(예: 640×640)로 맞춘다.
2. 백본(Backbone) 네트워크가 이미지의 특징을 추출한다.
3. 넥(Neck)이 여러 크기의 특징을 합쳐 작은 객체와 큰 객체를 모두 찾을 수 있게 한다.
4. 헤드(Head)가 바운딩 박스 좌표, 클래스, 신뢰도(confidence)를 출력한다.
5. NMS(Non-Maximum Suppression)로 같은 객체에 겹친 박스를 하나만 남긴다.

2.2 2 단계 탐지기와의 비교

구분	2 단계 (Faster R-CNN 등)	1 단계 (YOLO)
처리 방식	후보 영역 추출 후 분류	한 번에 위치와 클래스 예측
속도	느림	빠름 (실시간 가능)
적합한 용도	정밀 분석	영상·웹캠 실시간 처리

2.3 주요 버전

버전	발표	특징
YOLOv1	2016	최초의 YOLO, 실시간 탐지 개념 제시
YOLOv3	2018	다중 스케일 예측으로 작은 객체 성능 향상
YOLOv5	2020	PyTorch 기반, 사용이 쉬워 널리 보급
YOLOv8	2023	앵커 프리(anchor-free) 구조, Ultralytics 패키지로 제공
YOLO11	2024	v8 보다 적은 파라미터로 더 높은 정확도

본 과제는 자료가 많고 설치가 간단한 **YOLOv8** 을 사용한다.

3. 개발 환경

항목	내용
언어	Python 3.10 이상
탐지 라이브러리	Ultralytics (YOLOv8)
영상 처리	OpenCV
실행 환경	로컬 PC (Windows, i5-6600 CPU, GTX 1060 3GB GPU)
입력	도로 영상 파일(road.mp4) 또는 웹캠

설치 명령어는 한 줄이다.

```
pip install ultralytics opencv-python
# GPU 사용 시 (GTX 10 시리즈는 CUDA 12.6 빌드)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
```

4. 데이터셋과 탐지 클래스

별도 학습 없이 COCO 데이터셋(80 개 클래스, 약 33 만 장)으로 사전 학습된 가중치를 사용하고, 그중 교통 관련 5 개 클래스만 골라 탐지한다.

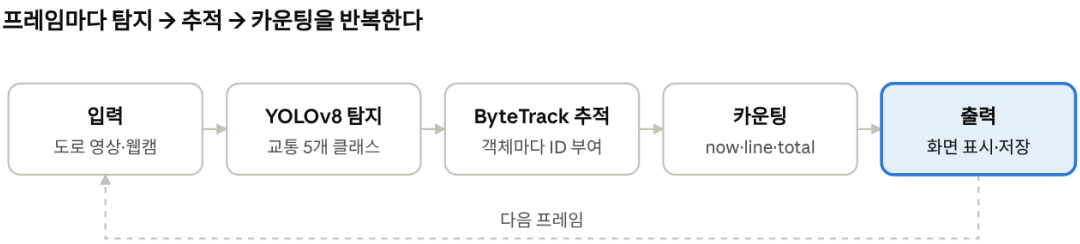
COCO ID	클래스	분류	화면 표시
0	person	보행자	보행자
2	car	차량	승용차
3	motorcycle	차량	오토바이
5	bus	차량	버스
7	truck	차량	트럭

테스트 영상은 서울 삼성역 교차로를 촬영한 YouTube 영상(Spotting Korea)의 일부 구간을 1080p 로 받아 사용한다. 한국 도로에 더 맞추려면 AI Hub 의 차량·보행자 데이터로 추가 학습(fine-tuning)할 수 있다.

5. 시스템 설계 및 구현

시스템은 영상 프레임마다 탐지와 추적을 거쳐, 현재 화면의 개수(now), 기준선 통과 개수(line), 누적 등장 개수(total)를 함께 표시한다.

5.1 처리 흐름



처리 흐름 · 5 단계, 프레임 반복

한 프레임의 처리가 끝나면 다음 프레임으로 돌아가며, 추적 ID 는 프레임이 바뀌어도 유지되어 누적 개수가 중복 없이 쌓인다.

5.2 핵심 기능

- **클래스 필터링**: `classes=[0, 2, 3, 5, 7]`로 보행자와 차량 4 종만 탐지한다.
- **작은 보행자 대응**: 높은 곳에서 찍은 영상은 사람이 약 20 픽셀로 작아, YOLOv8s 모델과 1280 추론 해상도를 쓰고 보행자 신뢰도 기준만 0.25 로 낮췄다 (차량은 0.4).
- **객체 추적(ByteTrack)**: `model.track(persist=True)`가 같은 객체에 프레임이 바뀌어도 같은 ID 를 붙인다.
- **기준선 통과 카운팅(line)**: 화면에 기준선을 긋고, 객체 중심점의 이동 경로가 선을 가로지르면 방향별로 한 번만 센다.
- **누적 카운팅(total)**: 클래스별로 처음 본 추적 ID 를 집합(set)에 넣어 센다.
- **화면 표시**: 클래스별 색상 박스, 신뢰도와 ID 라벨, 노란 기준선(통과 순간 초록으로 깜빡임), now·line·total 집계표, FPS 를 그린다.
- **결과 저장**: 처리된 영상을 `result.mp4` 로 저장한다.

5.3 핵심 코드

전체 코드는 `traffic_detect.py`(약 200 줄)이며, 아래는 기준선 통과 판정과 카운팅 부분이다.

```
def crossed(p_prev, p_now, a, b):
    """이동 경로(p_prev -> p_now)가 기준선 a-b 를 가로질렀으면 방향(+1/-1), 아니면 0"""
    s1, s2 = side(p_prev, a, b), side(p_now, a, b)
    if s1 == 0 or s1 * s2 > 0:
        return 0
    if side(a, p_prev, p_now) * side(b, p_prev, p_now) > 0:   # 선분 범위 밖
        return 0
    return 1 if s2 > 0 else -1

results = model.track(frame, classes=list(TARGET_CLASSES), conf=0.25, imgsz=1280,
                      persist=True, tracker="bytetrack.yaml")
for xyxy, cls, conf, tid in zip(...):
    if conf < (0.25 if cls == 0 else 0.4):     # 보행자는 낮게, 차량은 높게
        continue
    current[cls] += 1                           # now
    total_ids[cls].add(tid)                     # total
    if tid in last_pos and tid not in counted:
        d = crossed(last_pos[tid], center, A, B)
        if d:
            line_count[cls][d] += 1             # line
            counted.add(tid)
    last_pos[tid] = center
```

5.4 실행 방법

```
python traffic_detect.py                                    # road.mp4, yolov8s, 1280
python traffic_detect.py --source 0                         # 웹캠
python traffic_detect.py --model yolov8n.pt --imgsz 640     # 빠른 설정
python traffic_detect.py --line 0.1,0.6,0.9,0.6             # 가로 기준선 (x1,y1,x2,y2
비율)
python traffic_detect.py --baseline                         # 개선 기능 끄기 (6.5 절
비교용)
python traffic_detect.py --save ""                         # 영상 저장 끄기 (더 빠름)
```

6. 실험 결과 및 분석

모델이 클수록 정확도는 오르고 속도는 떨어지므로, 실시간(초당 30 프레임 이상)을 유지하는 가장 큰 모델을 고르는 것이 핵심이다.

6.1 공식 성능 (COCO val2017, 640×640)

모델	파라미터	mAP50-95	CPU 추론 (ONNX)
YOLOv8n	3.2M	37.3	80.4 ms
YOLOv8s	11.2M	44.9	128.4 ms
YOLOv8m	25.9M	50.2	234.7 ms

출처: Ultralytics YOLOv8 공식 문서 (80 개 클래스 전체 기준).

6.2 직접 측정한 결과

`evaluate.py` 로 삼성역 교차로 영상 150 프레임을 처리해 평균 FPS 를 쟀다 (탐지만 측정). GPU(GTX 1060)는 CPU 보다 3~13 배 빨랐고, 본 시스템 설정(YOLOv8s·1280)은 2.6 FPS 에서 27.8 FPS 로 올라 실시간(30 FPS)에 가까워졌다.

모델	CPU 640	CPU 1280	GPU 640	GPU 1280
YOLOv8n	21.8	6.6	70.7	46.6
YOLOv8s (본 시스템)	9.6	2.6	63.4	27.8
YOLOv8m	4.2	1.1	37.0	14.4

추적·화면 표시·영상 저장까지 포함한 전체 처리(90 초, 2,698 프레임)는 CPU 19 분(약 2.4 FPS), GPU 238 초(약 11 FPS)가 걸렸고, 탐지·카운팅 결과는 두 장치에서 같았다.

정확도(mAP) 직접 측정

`evaluate.py` 로 COCO val2017(5,000 장)에서 교통 관련 5 개 클래스(사람·승용차·오토바이·버스·트럭)의 mAP 를 재고, 같은 실행에서 GPU 탐지 FPS 도 함께 재다(2026-10-14). 모델이 클수록 mAP 는 오르고 속도는 떨어졌으며, s 는 n 보다 mAP 가 0.075 높으면서 속도는 비슷했다.

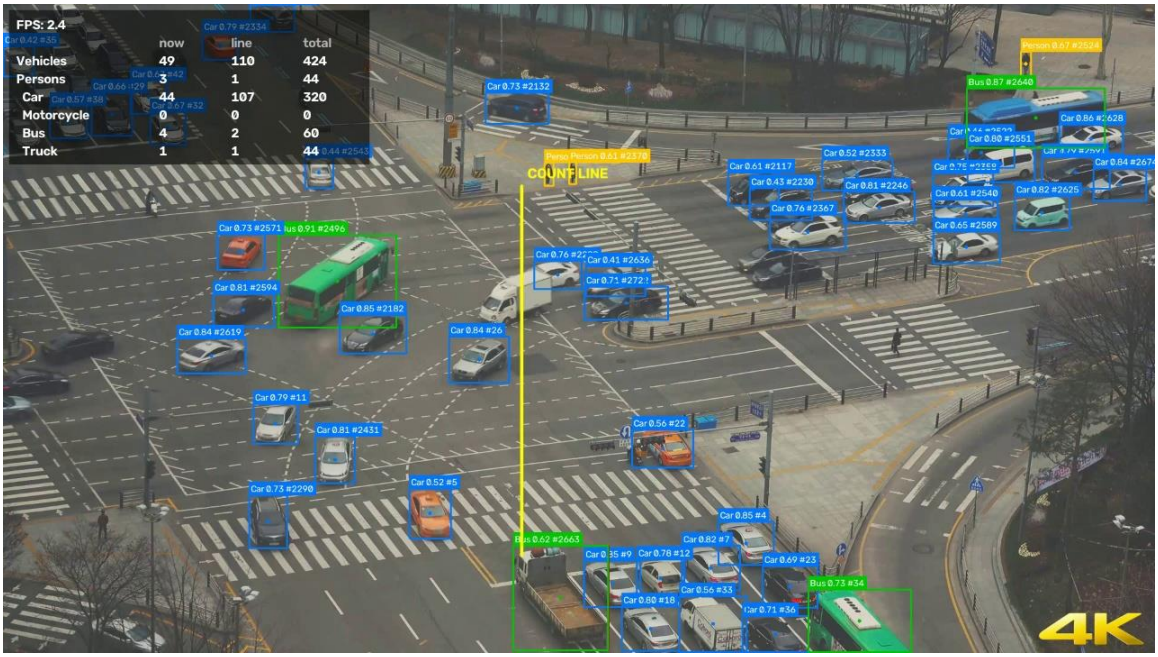
모델	해상도	mAP50-95	mAP50	GPU FPS
YOLOv8n	640	0.438	0.626	60.5
YOLOv8s	640	0.513	0.708	57.9
YOLOv8s (본 시스템)	**1280**	**0.438**	**0.670**	**25.5**
YOLOv8m	640	0.567	0.759	34.2

COCO 에서는 YOLOv8s·1280(0.438)이 640(0.513)보다 낮다. 모델이 640 으로 학습됐고 COCO 물체는 대부분 중간 크기 이상이기 때문이다. 본 시스템이 1280 을 쓰는 근거는 테스트 영상의 작은 보행자다. `road.mp4` 에서 고르게 뽑은 60 프레임을 YOLOv8s 로 탐지하면 사람이 640 에서 12 개(높이 40 픽셀 미만 5 개), 1280 에서 190 개(100 개) 잡혔다(`compare_resolution.py`, 2026-10-15).

측정 환경: CPU Intel Core i5-6600, 메모리 16GB, GPU NVIDIA GeForce GTX 1060 3GB, PyTorch 2.14 (CUDA 12.6), Python 3.14. 6.1 절 공식 mAP 는 80 개 클래스 기준이고 위 직접 측정은 5 개 클래스 평균이라 두 값은 직접 비교하지 않는다.

6.3 탐지 화면

삼성역 교차로 영상 90 초(2,698 프레임)을 YOLOv8s·1280 설정으로 처리한 `result.mp4` 의 장면이다. 왼쪽 위 표가 now·line·total, 가운데 노란 세로선이 기준선이다.





최종 집계 (90 초)

구분	line (기준선 통과)	total (누적 고유 ID)
승용차	146 (방향별 75 / 71)	531
버스	3	125
트럭	2	97
오토바이	0	0
보행자	1	75

6.4 분석

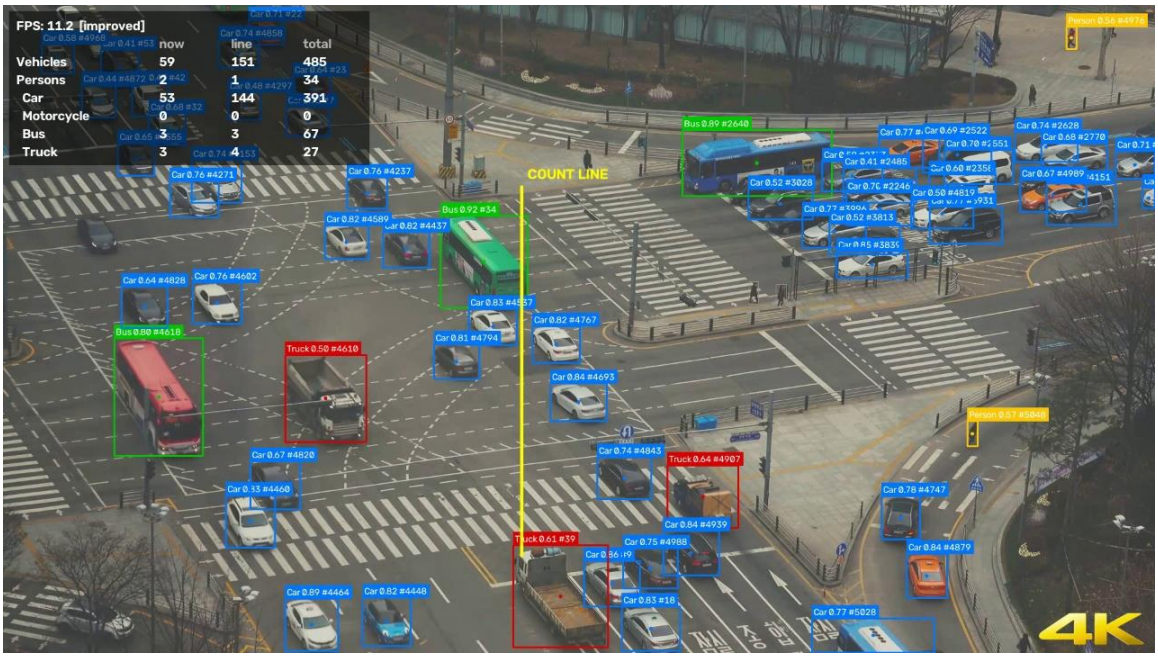
- **잘 된 점**: 화면에 동시에 있는 차량 50~60 대를 대부분 탐지했고, 기준선 통과 승용차는 양방향 75 대·71 대로 고르게 집계됐다.
- **작은 보행자**: 사람이 약 20 픽셀로 작아 기본 설정(YOLOv8n·640)에서는 한 장면에 1 명만 잡혔다. YOLOv8s·1280 으로 바꾸자 5 명으로 늘었지만, 횡단보도 위 보행자는 여전히 자주 놓쳤다.
- **오탐지**: 신호등이 보행자(Person)로 잡히는 경우가 많았다. 보행자 신뢰도 기준을 0.25 로 낮춘 대가다. (6.5 절에서 개선)
- **클래스 혼동**: 짐칸 트럭이 버스로 잡히는 등, 같은 차가 프레임마다 클래스가 바뀌었다.
- **total 이 부풀려짐**: 차가 다른 차나 기둥에 가려졌다 나타나면 새 ID 가 붙고, 클래스가 바뀌면 다른 클래스에도 집계돼 버스 total 이 125 까지 올랐다. 실제 교통량은 line 값이 더 믿을 만하다. (6.5 절에서 개선)
- **속도**: CPU(i5-6600)에서는 약 2.4 FPS 로 90 초 영상에 19 분이 걸렸지만, GPU(GTX 1060)로 바꾸자 약 11 FPS, 238 초로 줄었다. 탐지만 보면 27.8 FPS 이므로, 남은 병목은 추적·화면 그리기·영상 저장이다.

6.5 개선 실험: 오탐지와 total 부풀림 줄이기

6.4 절에서 찾은 문제 세 가지를 고친 개선 버전을 같은 90 초 영상에 돌린 결과, 차량 total 은 753 에서 485 로(36% 감소), 보행자 total 은 75 에서 35 로(53% 감소) 줄었고 교통량(line)은 그대로 유지됐다.

문제	개선 방법
신호등이 보행자로 오탐지	**위치 기반 정지 물체 필터**: 화면을 32 픽셀 칸으로 나눠, 20 초 이후 지금까지 프레임의 15% 이상 '보행자'가 나온 칸의 보행자는 제외한다. 신호등은 탐지가 끊겼다 이어지며 ID 가 계속 바뀌므로 ID 가 아니라 위치로 판단한다.
트럭·버스 클래스 혼동	**클래스 다수결**: 추적 ID 별로 지금까지 가장 많이 나온 클래스를 쓴다.
잠깐 나타난 ID 로 total 부풀림	**짧은 추적 제거**: 5 프레임보다 짧게 보인 ID 는 total 에서 뺀다.

구분	기본 line	개선 line	기본 total	개선 total
승용차	146	144	531	391
버스	3	3	125	67
트럭	2	4	97	27
차량 합계	151	151	753	485
보행자	1	1	75	35



- **정지 물체 필터**: 21 개 ID 가 정지 물체로 판정됐다. 6.3 절 두 번째 캡처와 같은 장면에서 신호등 5 개 중 3 개가 걸러졌고, 영상 후반에 처음 나타난 2 개는 비율이 기준에 못 미쳐 남았다.
- **클래스 다수결**: 차량 line 합계는 151 로 같고, 승용차 2 대가 트럭으로 다시 분류됐다. 버스 total 은 125 에서 67, 트럭 total 은 97 에서 27 로 줄었다.
- **한계**: 신호등 위치의 '보행자' 비율은 18~45%였지만, 실제 보행자가 신호를 기다리는 모토이도 28%가 나왔다. 그래서 이 필터는 한자리에 오래 서 있는 진짜 보행자도 뺄 수 있다.
- **속도**: 영상을 저장하지 않는 같은 조건에서 기본 16.7 FPS, 개선 16.8 FPS 로 차이가 없었다. 1080p 결과 영상을 저장하면 약 11.9 FPS 로 떨어져, 속도를 가장 많이 잡아먹는 것은 영상 저장이었다.

개선 기능은 PC 용 `traffic_detect.py` 에 기본으로 들어 있고(`--baseline` 으로 끌 수 있음), 이 실험은 Colab 노트북 `TrafficLens_Colab.ipynb` 의 실험 2·3 으로 그대로 재현할 수 있다.

7. 결론 및 향후 개선점

YOLOv8 과 ByteTrack 을 결합해, 별도 학습 없이도 도로 영상에서 차량과 보행자를 탐지하고 기준선을 지나는 교통량을 종류별로 셀 수 있음을 확인했다. 처리 속도는 GPU 로 바꾸자 CPU 대비 약 4.8 배 빨라졌지만, 전체 처리 기준 약 11 FPS 로 실시간(30 FPS)에는 아직 못 미쳤다.

7.1 결론

- 사전 학습 모델과 클래스 필터만으로 교통 객체 5 종을 탐지할 수 있었다.
- 기준선 통과 카운팅으로 90 초 동안 승용차 146 대의 교통량을 방향별로 집계했다. 누적 ID(total)는 가림과 클래스 혼동으로 부풀려졌지만, 클래스 다수결·짧은 추적 제거·위치 기반 정지 필터로 차량 36%, 보행자 53%를 줄였다.
- 높은 곳에서 찍은 영상의 작은 보행자는 모델과 해상도를 키워야 잡혔고, 그만큼 속도가 떨어졌다 (CPU 2.4 FPS → GPU 약 11 FPS).

7.2 향후 개선점

1. **실시간 처리**: GPU 적용으로 탐지는 27.8 FPS 까지 올랐으므로, 영상 저장을 끄거나 화면 그리기를 줄여 전체 처리도 실시간에 가깝게 만든다.
2. **보행자 오탐지 줄이기**: 위치 기반 필터는 신호등 5 개 중 3 개만 걸렀다. 후반에 나타난 신호등까지 잡도록 판단 방식을 다듬고, 기다리는 보행자와 구분하려면 높은 시점의 보행자 데이터로 추가 학습한다.
3. **크게 나눠 탐지(타일링)**: 화면을 여러 조각으로 나눠 탐지하면 작은 보행자를 더 잘 찾을 수 있다.
4. **영상 저장 속도**: 1080p 결과 영상 저장만으로 16.8 FPS 가 11.9 FPS 로 떨어진다. 저장 해상도를 낮추거나 별도 스레드에서 저장해 속도를 되찾는다.
5. **한국 도로 데이터 학습**: AI Hub 데이터로 fine-tuning 해 국내 버스·트럭 인식률을 높인다.

참고문헌

1. Redmon, J. et al. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR.
2. Jocher, G. et al. (2023). Ultralytics YOLOv8. Ultralytics 공식 문서.
3. Zhang, Y. et al. (2022). ByteTrack: Multi-Object Tracking by Associating Every Detection Box. ECCV.
4. Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV.
5. AI Hub 차량·보행자 관련 데이터셋.
6. Spotting Korea. 삼성역 교차로 자동차 영상 4K Spotting Seoul. YouTube. (테스트 영상)

부록. 발표 자료

발표는 슬라이드 20 장, 약 10 분 15 초 분량이다. 같은 대본이 아래 자료에 함께 들어 있다(2026-10-16 기준).

자료	파일 · 위치	내용
발표 슬라이드	`TrafficLens_발표_20 장.pptx` · `.pdf`	20 장. 장마다 발표자 노트(대본, 동작, 넘기는 말, 키워드, 시간)
발표 준비 자료	`TrafficLens_presenter_kit.docx` · `.pdf`	1 부 대본, 2 부 기억할 키워드와 숫자, 3 부 예상 질문과 답변 31 개
소개 사이트	trafficlens.maxfinup.com/slides	슬라이드 20 장과 장별 대본

발표 16 장의 정답 비교 결과(기준선 통과 수 오차)는 아직 측정 중이며, 측정이 끝나면 6 장과 발표 자료에 함께 반영한다.

프로젝트 진행 과정(2026.10.13 ~ 10.31, 1~6 차 일지)은 프로젝트 작업일지에 정리했다: 소개 사이트 자료실에서 PDF · Word 내려받기 · 작업일지 원본 문서.